*Core methodology, baseline modelling, and prototype development*


## Abstract

This project investigates lightweight speech command recognition for Arabic and English, with emphasis on generalization to unseen speakers and realistic operating conditions. The study combines the Arabic Speech Commands dataset (12,000 one-second recordings, 40 Modern Standard Arabic command classes, 30 speakers) with Google Speech Commands (GSC; preferably v0.02, subject to local verification). The methodological core is a controlled comparison of language-specific baselines and a shared multilingual model, followed by data-augmentation and optional text-to-speech (TTS) experiments. Three model families are proposed as baselines: Depthwise Separable Convolutional Neural Network (DS-CNN), Convolutional Recurrent Neural Network (CRNN), and Keyword Transformer (KWT-1). Audio is standardized to 16 kHz, mono, one-second clips and represented using 40-bin Log-Mel filterbanks or, in a secondary experiment, Mel-Frequency Cepstral Coefficients (MFCCs). Evaluation uses speaker-disjoint splits, Accuracy, macro-F1, per-class Precision/Recall, confusion matrices, and false-acceptance/false-rejection measures for negative classes. The protocol explicitly distinguishes evidence supported by the supplied datasets and reference studies from proposed experiments that must be verified empirically.


## 1. Problem Definition and Scope

The task is formulated primarily as closed-set and open-set Keyword Spotting (KWS): given a short audio segment x, predict one of the supported command labels, or reject it as unknown/silence. It is not a general-purpose Automatic Speech Recognition (ASR) task. The target application is a Speech-Activated System (SAS) that recognizes a predefined command vocabulary in Arabic and English and maps recognized labels to application actions.

The research scope is:

- establish language-specific baseline performance on Arabic and English;
- assess whether joint training on a shared label space improves or harms performance;
- quantify generalization to speakers not used during training or model selection;
- assess robustness to background noise and temporal variation;
- estimate the accuracy–efficiency trade-off for edge deployment;
- optionally evaluate whether carefully validated TTS-generated Arabic data improves robustness.

The central research question is: Do lightweight KWS models generalize to unseen speakers in Arabic and English, and to what extent do multilingual joint training and data augmentation improve that generalization?

## 2. Research Questions

RQ1. How do DS-CNN, CRNN, and KWT-1 compare on Arabic-only and English-only command recognition?

RQ2. Does a shared bilingual model improve performance on the shared command vocabulary, or does it introduce negative transfer due to dataset imbalance and recording-domain differences?

RQ3. How much does performance change when evaluation is performed on speakers excluded from training and validation?

RQ4. How robust are the models to background noise, temporal shifts, and moderate speaking-rate variation?

RQ5. What accuracy, memory, computational-cost, and latency trade-offs arise for lightweight architectures?

RQ6 (optional). Does TTS-based augmentation improve performance on real human recordings, rather than only on synthetic evaluation data?

## 3. Datasets and Data Governance

### 3.1 Arabic Speech Commands

The dataset description reports 12,000 WAV files for 40 Modern Standard Arabic command classes, with 30 speakers and 10 repetitions per word per speaker. Each class has 300 recordings. Files are described as one second long, mono, 16 kHz, with a background_noise directory of approximately 49 MB. The provided split is speaker-disjoint: Train 7,200 clips from 18 speakers, Validation 2,400 clips from 6 speakers, and Test 2,400 clips from 6 speakers. The stated licence is CC BY 4.0. The small number of speakers and the restriction to Modern Standard Arabic limit claims about dialectal generalization. (Ghandoura et al., 2021).

### 3.2 Google Speech Commands (GSC)

Use the official GSC v0.02 if that is the version actually present in the downloaded package. The commonly reported dataset counts are approximately 105,829 files and 35 words for v0.02, compared with 64,727 files and 30 words for v0.01; verify these counts against the exact downloaded release. Recordings are WAV, mono, 16 kHz, and no longer than approximately one second; the official split lists should be preserved where available. The Kaggle mirror's exact version and contents were not verified in the supplied notes, so record the downloaded archive name, version, checksum, class folders, and presence of validation/testing lists and background-noise files before experiments. Do not assume the Kaggle mirror is identical to the official release. (Warden, 2018).

### 3.3 Label alignment

The dataset analysis identifies 19 lexical matches between the Arabic vocabulary and GSC: digits zero–nine, yes/no, up/down/left/right, forward/backward, and stop. Use a documented mapping table and do not merge labels merely because their meanings are similar. For example, Arabic “تعطيل/تفعيل” are semantically related to “on/off” but are not literal lexical matches. The 40 Arabic classes that do not have a suitable English counterpart remain language-specific labels in the multilingual label space.

### 3.4 Negative classes

The Arabic corpus has 40 command classes but does not, according to the dataset description, contain explicit silence and unknown classes. Construct these classes from background-noise segments and out-of-vocabulary (OOV) words, including unused GSC words where appropriate. Ensure negative examples are diverse and do not accidentally contain target commands. Define the sampling policy and class balance before training.

### 3.5 Dataset manifest and audit

Create a manifest with at least: file path, source dataset, speaker ID if available, language, original label, mapped label, duration, sample rate, channel count, split, and any augmentation lineage. Validate file readability, clipping, silence, duration, sample rate, channels, and label/path consistency. Report corrupted or excluded files and reasons. Store dataset version, licence, download date, and cryptographic checksums in the experiment log.

## 4. Experimental Design and Leakage Prevention

Use the published speaker-disjoint Arabic split and the official GSC validation/testing lists when the package supports them. Never randomly split individual clips if this permits the same speaker to appear across training and evaluation. The primary test set must remain untouched until model selection and threshold tuning are complete.

Because the Arabic and English corpora differ substantially in size, speaker count, recording conditions, and label distribution, report results separately for each language before reporting any pooled bilingual score. Create a separate team-recorded challenge set, if feasible, from new speakers not represented in either training set. Record multiple repetitions per command and include natural variation in distance, volume, accent, and realistic background conditions. This external set is particularly important because the Arabic set contains only 30 speakers and only Modern Standard Arabic.

Recommended protocol:

1. Freeze the defined splits and document their provenance.
2. Fit all learned preprocessing or normalization parameters on training data only.
3. Tune architecture, augmentation strength, class weighting, and rejection thresholds on validation data only.
4. Evaluate once on the fixed test sets and the separate challenge set.
5. Use the same evaluation protocol and preprocessing for all model comparisons.
6. Report multiple random seeds when feasible; provide mean and standard deviation or confidence intervals.
7. If the same speaker appears in multiple source corpora and identity can be reliably established, prevent cross-corpus speaker leakage. Do not attempt to identify anonymous GSC contributors.

## 5. Audio Preprocessing

### 5.1 Canonical representation

Decode audio consistently, convert to mono, resample to 16,000 Hz if required, and pad or crop to a one-second window (16,000 samples). Keep a record of every resampling or cropping operation. For short clips, use deterministic padding; for longer clips, use a documented crop policy. Avoid removing silence indiscriminately because silence boundaries can be relevant to streaming KWS and negative-class detection.

### 5.2 Signal checks

For each file, inspect duration, sample rate, channel count, peak amplitude, clipping, RMS energy, and abnormal silence. Reject only files that meet predeclared corruption criteria; log all exclusions. Amplitude normalization may be evaluated as an ablation rather than applied blindly, since absolute energy can be informative for some deployment settings.

### 5.3 Feature extraction

Primary representation: 40-bin Log-Mel filterbank features. A reproducible initial configuration is a 25 ms analysis window and 10 ms hop at 16 kHz, with a Hann window, power spectrum, Mel filterbank, and log compression. The precise FFT size and implementation should be fixed in the experiment configuration and held constant across models. Secondary representation: MFCCs, evaluated as a controlled ablation. The supplied SM-STMC study uses a 1024-sample window and 256-sample hop for its own setup; this is a reference configuration, not a mandatory setting for this project. (Davis & Mermelstein, 1980).

### 5.4 Noise and augmentation

Apply augmentation only to training data: mix with background noise at sampled signal-to-noise ratios (SNRs), small temporal shifts, modest speed perturbation (for example 0.9× and 1.1×), and optional gain variation. Consider SpecAugment on feature tensors. Set ranges through validation experiments and avoid unrealistic transformations that change the phonetic identity of a command. Do not augment validation or test data in the primary evaluation; create a separately labelled robustness test if testing corrupted conditions. (Park et al., 2019).

## 6. Model Architectures

### 6.1 DS-CNN

Use depthwise separable convolutions to reduce multiply–accumulate operations and parameter count. It is the primary edge-oriented baseline. Document input feature dimensions, channel widths, kernel sizes, strides, normalization, activation functions, pooling, classifier head, parameter count, and estimated computational cost. (Zhang et al., 2017).

### 6.2 CRNN

Use convolutional layers to learn local time-frequency patterns followed by a recurrent layer (for example, GRU or LSTM) to model temporal dependencies. A bidirectional recurrent layer is acceptable for offline one-second classification, but a strictly causal/unidirectional design should be considered if streaming latency is a requirement. The supplied benchmark reports strong results for CRNN-L, but this is evidence from English GSC-related data and does not guarantee the same ranking for Arabic.

### 6.3 KWT-1

Use a compact Keyword Transformer as a higher-capacity comparison model. Treat it as an accuracy-oriented reference and measure inference cost rather than assuming it is suitable for a microcontroller. Document tokenization/patching of feature frames, embedding dimension, number of attention heads, transformer depth, positional encoding, and classifier head. (Berg et al., 2021).

### 6.4 Model controls

Use identical data splits and feature extraction across architectures. Compare parameter count, model size, latency, and performance. Include a fixed-seed training protocol and save the best validation checkpoint. A plain DNN may be included as a diagnostic baseline, but it should not replace the stronger planned architectures; the benchmark reports a substantial generalization drop for a simple DNN on newly recorded speakers.

## 7. Training Protocol

Train separate Arabic-only and English-only models first. These provide language-specific baselines and reveal whether a model's performance is dominated by corpus properties. Then train a bilingual model

under a clearly defined label mapping. In the primary shared-vocabulary experiment, use the 19 lexical matches; in an optional full-vocabulary experiment, retain all 40 Arabic labels and supported English labels as distinct classes.

Use cross-entropy for single-label classification. Select the optimizer (for example Adam or AdamW), learning rate, batch size, scheduler, weight decay, and epoch limit using validation performance. Apply early stopping based on validation macro-F1 or a predeclared primary metric. If class imbalance occurs, compare balanced sampling or class-weighted loss against unweighted training. Do not choose hyperparameters based on test performance.

To control the multilingual data imbalance, compare at least:

A. natural sampling;

B. balanced or temperature-based sampling by language/class;

C. optionally, loss weighting.

Report each strategy explicitly. Keep augmentation lineage and random seeds reproducible. Store software versions, configuration files, checkpoints, and logs.

## 8. Bilingual and Command-Decision Pipeline

The proposed system has two logical stages:

Stage 1 — Acoustic KWS: map the audio feature sequence to a posterior distribution over command classes, including unknown and silence.

Stage 2 — Decision and action mapping: apply a confidence threshold and optional temporal smoothing/debounce logic, then map the accepted class to a safe application action.

For bilingual operation, choose one of two explicit designs:

- Language-conditioned mode: the user or application selects Arabic or English, reducing label ambiguity.
- Shared bilingual mode: a single classifier predicts from a unified label inventory, with language-specific labels kept distinct except for intentionally shared lexical labels.

Do not describe this pipeline as translation unless a translation module is actually implemented. KWS classifies predefined commands; it does not necessarily transcribe arbitrary speech. If the application requires free-form utterances, that is a separate ASR extension and should be evaluated separately.

Use a minimum confidence threshold selected on validation data, and report false acceptance rate (FAR) and false rejection rate (FRR). In streaming mode, consider consecutive-frame agreement, hysteresis, cooldown intervals, and delayed endpointing to reduce repeated or premature actions. These are design options to test, not assumptions of the current datasets.

## 9. Experimental Matrix and Ablation Studies

Run experiments in a staged design:

Core E1. Arabic-only: DS-CNN and CRNN. Add KWT-1 only if time and compute permit.

Core E2. English-only: DS-CNN and CRNN. Add KWT-1 only if time and compute permit.

E3. Bilingual model on the 19 shared labels.

E4. Optional full-vocabulary bilingual model with distinct language-specific classes.

Core E5. Focused augmentation comparison: no augmentation versus noise and temporal shift. Speed perturbation and combined augmentation are optional extensions.

E6. Feature ablation: Log-Mel versus MFCC.

E7. Negative-class ablation: without versus with explicit silence/unknown training examples.

E8. Robustness evaluation across predefined SNR bands and temporal offsets.

Optional E9 (later phase). TTS augmentation for Arabic, evaluated exclusively on real human speech in the primary test.

Optional E10 (later phase). Efficiency evaluation: float model versus quantized model (for example int8) where the deployment stack supports it.

Change one factor at a time where possible. For each experiment, state the research question, controlled variables, changed variable, validation rule, and final test protocol.

## 10. Evaluation Metrics and Statistical Reporting

Primary metrics:

- Accuracy: proportion of all test examples classified correctly.
- Macro-F1: unweighted average of class-wise F1, useful when classes or language subsets are imbalanced.
- Per-class Precision and Recall, plus a confusion matrix.
- FAR: fraction of negative examples incorrectly accepted as commands.
- FRR: fraction of target-command examples rejected or missed at the chosen threshold.
- For streaming use, false triggers per hour and detection latency are recommended.

Efficiency metrics:

- parameter count and serialized model size;
- peak runtime memory;
- multiply–accumulate operations or another clearly defined compute proxy;
- median and high-percentile inference latency on the target device;
- energy consumption if reliable measurement hardware is available.

For any ASR extension, report Word Error Rate (WER); for Arabic–English code-switching or orthographic variation, consider adding BERTScore as a complementary semantic metric. WER and BERTScore are not substitutes for command classification metrics.

Report metrics separately by language, model, class, noise condition, and speaker group. Include confidence intervals or variation across seeds where possible. Compare systems using paired predictions on the same test items. State the confidence-interval method and any statistical test used. Avoid claiming statistical significance based on point estimates alone.

## 11. Error Analysis and Threats to Validity

Inspect false positives and false negatives, especially acoustically or semantically confusable pairs such as right/left, up/down, open/close, and Arabic equivalents. Review confusion matrices, confidence scores, SNR, speaker, and recording source. Listen to misclassified clips under appropriate data-use permissions; do not infer speaker identity from anonymous datasets.

Key threats:

- limited Arabic speaker count and Modern Standard Arabic-only coverage;
- potential mismatch between the Kaggle mirror and official GSC release;
- dataset imbalance and different recording conditions between Arabic and English;
- vocabulary mismatch: only 19 labels are described as lexical matches;
- negative classes may not reflect real-world background speech unless deliberately collected;
- synthetic data may improve synthetic-domain performance without improving human-speech performance;
- results on one-second clips may not transfer to continuous streaming audio;
- very high benchmark results can be inflated by leakage, duplicate recordings, or overly easy test conditions unless splits and provenance are audited.

Mitigations include speaker-disjoint splits, a newly recorded challenge set, transparent class mapping, held-out noise conditions, reproducible manifests, and cautious interpretation of TTS experiments.

## 12. Reproducibility, Ethics, and Data Management

Maintain a version-controlled experiment repository containing preprocessing scripts, dataset manifests, label maps, training configurations, evaluation scripts, and environment specifications. Record random seeds, library versions, model checkpoints, hardware, training duration, and exact command-line settings. Preserve dataset licences and citations; the dataset notes state CC BY 4.0 for both corpora, but verify the licence terms for the exact downloaded artifacts and any third-party mirror.

For newly collected speech, obtain informed consent, explain the purpose and retention period, avoid collecting unnecessary personal information, restrict access, and provide a deletion route where applicable. Do not attempt to identify speakers in datasets whose terms prohibit it. If recordings include sensitive commands or personal content, minimize retention and access. TTS-generated data must be labelled as synthetic and kept traceable to the generating model/configuration.

## 13. Proposed End-to-End Pipeline

1. Verify dataset versions, licences, directory structures, and split files.
2. Build and validate the metadata manifest.
3. Audit signal quality and standardize audio to mono, 16 kHz, one second.
4. Define label mappings and add explicit silence/unknown negative examples.
5. Extract 40-bin Log-Mel features; preserve MFCC as an ablation.
6. Train Arabic-only and English-only DS-CNN, CRNN, and KWT-1 baselines.
7. Select checkpoints and thresholds using validation data only.
8. Train the bilingual shared-vocabulary model and compare sampling strategies.
9. Run augmentation, feature, negative-class, robustness, and efficiency ablations.
10. Evaluate on fixed speaker-disjoint test sets and, ideally, a new-speaker challenge set.
11. Perform error analysis and report accuracy, macro-F1, FAR/FRR, confusion matrices, and efficiency.
12. Document limitations and release only artifacts permitted by dataset licences and participant consent.

## References


Berg, A., O'Connor, M., & Cruz, M. T. (2021). Keyword Transformer: A self-attention model for keyword spotting. In Proceedings of Interspeech 2021 (pp. 4249–4253). https://doi.org/10.21437/Interspeech.2021-1286

Davis, S. B., & Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. IEEE Transactions on Acoustics, Speech, and Signal Processing, 28(4), 357–366. https://doi.org/10.1109/TASSP.1980.1163420

Ghandoura, A., Hjabo, F., & Al Dakkak, O. (2021). Building and benchmarking an Arabic Speech Commands dataset for small-footprint keyword spotting. Engineering Applications of Artificial Intelligence, 102, Article 104267. https://doi.org/10.1016/j.engappai.2021.104267

Park, D. S., Chan, W., Zhang, Y., Chiu, C.-C., Zoph, B., Cubuk, E. D., & Le, Q. V. (2019). SpecAugment: A simple data augmentation method for automatic speech recognition. In Proceedings of Interspeech 2019 (pp. 2613–2617). https://doi.org/10.21437/Interspeech.2019-2680

Sainath, T. N., & Parada, C. (2015). Convolutional neural networks for small-footprint keyword spotting. In Proceedings of Interspeech 2015 (pp. 1478–1482). https://doi.org/10.21437/Interspeech.2015-352

Warden, P. (2018). Speech Commands: A dataset for limited-vocabulary speech recognition. arXiv. https://arxiv.org/abs/1804.03209

Zhang, Y., Suda, N., Lai, L., & Chandra, V. (2017). Hello Edge: Keyword spotting on microcontrollers. arXiv. https://arxiv.org/abs/1711.07128